Disease Data Management and Analytics System

This presentation outlines the development of a comprehensive disease data management system aimed at improving public health outcomes. We cover problem definition, database design, implementation, analytics, and modern cloud deployment strategies. The system supports tracking diseases, analyzing treatment effectiveness, and identifying risk factors to enable early intervention and informed decision-making.

Defining the Business Problem and Data Model

The goal is to enhance public health by tracking diseases, analyzing treatment effectiveness, and studying patient demographics and risk factors. Public health agencies require a reliable database system to monitor racial and geographic disparities and support operational and strategic analytics.

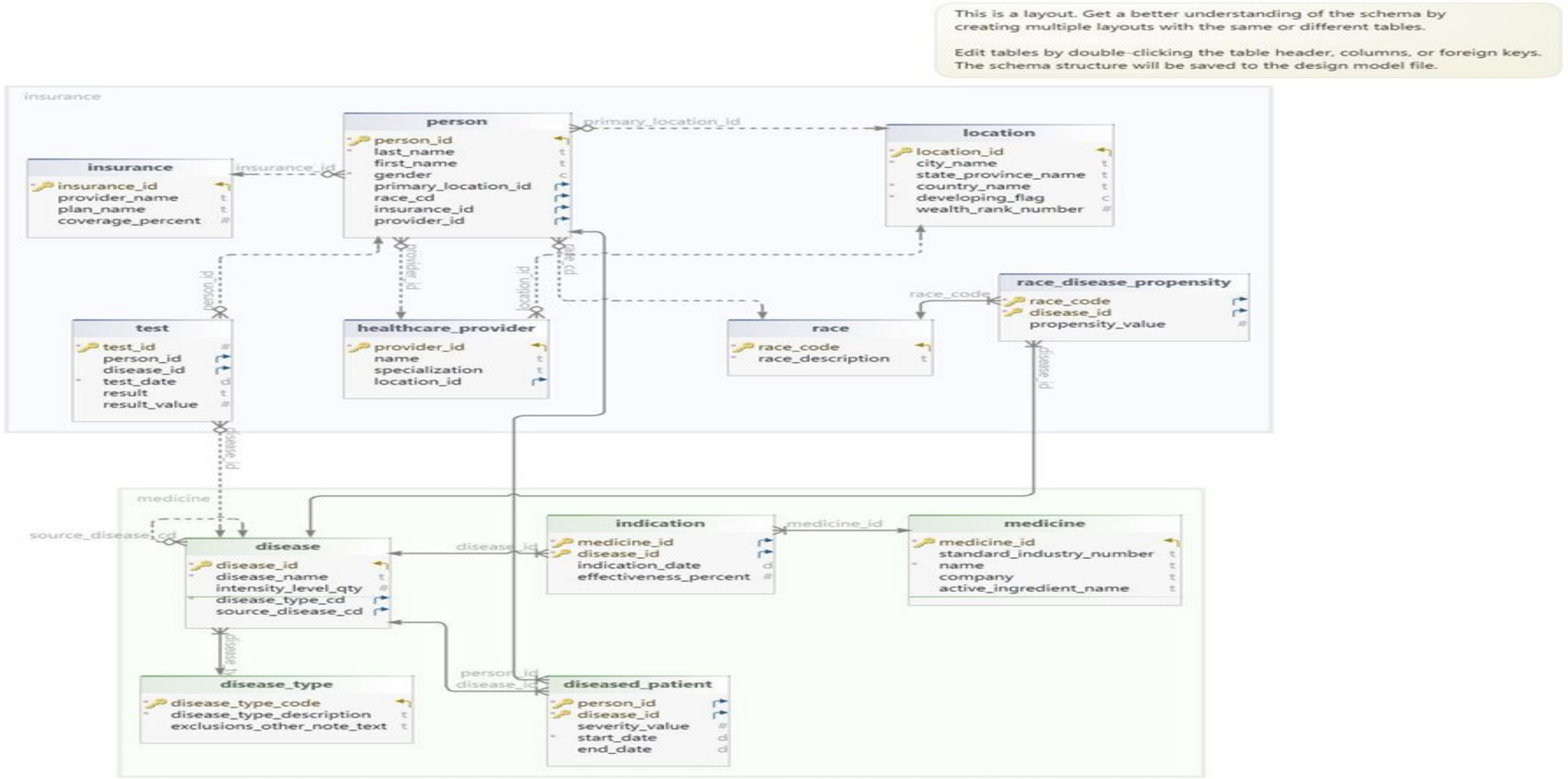
Business Goal

Improve health outcomes via disease tracking and analytics.

Data Model

Relational schema with entities for patients, diseases, tests, and locations.

ER DIAGRAM



The relational **ER diagram** and data dictionary define the schema, capturing entities like patients, diseases, tests, and locations to structure the data effectively.

DATA DICTIONARY

Table disease

Idx	Column Name	Data Type
*🔑↰	disease_id	serial
*	disease_name	varchar(100)
	intensity_level_qty	integer DEFAULT 1
*↱	disease_type_cd	char(5)
↱	source_disease_cd	integer
Indexes		
🔑	disease_pkey	Primary Key ON disease_id
Foreign Key		
↱	disease_disease_type_cd_fkey	disease_type_cd ↗ ❑ disease_type(disease_type_code)
↱	disease_source_disease_cd_fkey	source_disease_cd ↗ ❑ disease(disease_id)
Referring Foreign Key		
↰	disease_source_disease_cd_fkey	disease_id ↙ ❑ disease(source_disease_cd)
↰	diseased_patient_disease_id_fkey	disease_id ↙ ❑ diseased_patient
↰	indication_disease_id_fkey	disease_id ↙ ❑ indication
↰	race_disease_propensity_disease_id_fkey	disease_id ↙ ❑ race_disease_propensity
↰	test_disease_id_fkey	disease_id ↙ ❑ test
Constraints		
	disease_intensity_level_qty_check	((intensity_level_qty >= 1) AND (intensity_level_qty <= 10))

Table diseased_patient

Idx	Column Name	Data Type
*🔑↱	person_id	integer
*🔑↱	disease_id	integer
	severity_value	integer DEFAULT 1
*	start_date	date
	end_date	date
Indexes		
🔑	diseased_patient_pkey	Primary Key ON person_id, disease_id
Foreign Key		
↱	diseased_patient_person_id_fkey	person_id ↗ ❑ person
↱	diseased_patient_disease_id_fkey	disease_id ↗ ❑ disease
nstraints		
	diseased_patient_severity_value_check	((severity_value >= 1) AND (severity_value <= 10))

Table test

Idx	Column Name	Data Type
*🔑	test_id	serial
↱	person_id	integer
↱	disease_id	integer
*	test_date	date
	result	varchar(50)
	result_value	double precision
Indexes		
🔑	test_pkey	Primary Key ON test_id
Foreign Key		
↱	test_person_id_fkey	person_id ↗ ❑ person
↱	test_disease_id_fkey	disease_id ↗ ❑ disease

Creating Tables

Query | Query History

```sql
-- 1. Disease Type
CREATE TABLE disease_type (
    disease_type_code CHAR(5) PRIMARY KEY,
    disease_type_description VARCHAR(1000) NOT NULL,
    exclusions_other_note_text VARCHAR(2000)
);

-- 2. Disease
CREATE TABLE disease (
    disease_id SERIAL PRIMARY KEY,
    disease_name VARCHAR(100) NOT NULL,
    intensity_level_qty INT DEFAULT 1 CHECK (intensity_level_qty BETWEEN 1 AND 10),
    disease_type_cd CHAR(5) NOT NULL REFERENCES disease_type(disease_type_code),
    source_disease_cd INT REFERENCES disease(disease_id)
);

-- 3. Race
CREATE TABLE race (
    race_code CHAR(5) PRIMARY KEY,
    race_description VARCHAR(100) NOT NULL
);

-- 4. Location
CREATE TABLE location (
    location_id SERIAL PRIMARY KEY,
    city_name VARCHAR(100) NOT NULL,
    state_province_name VARCHAR(100),
    country_name VARCHAR(100) NOT NULL,
    developing_flag CHAR(1) NOT NULL CHECK (developing_flag IN ('Y', 'N')),
    wealth_rank_number INT CHECK (wealth_rank_number BETWEEN 1 AND 10)
);

-- 5. Insurance
CREATE TABLE insurance (
    insurance_id SERIAL PRIMARY KEY,
    provider_name VARCHAR(150),
    plan_name VARCHAR(150),
    coverage_percent FLOAT CHECK (coverage_percent BETWEEN 0 AND 100)
);
```

Query | Query History

```sql
-- 6. Healthcare Provider
CREATE TABLE healthcare_provider (
    provider_id SERIAL PRIMARY KEY,
    name VARCHAR(150),
    specialization VARCHAR(100),
    location_id INT REFERENCES location(location_id)
);

-- 7. Person
CREATE TABLE person (
    person_id SERIAL PRIMARY KEY,
    last_name VARCHAR(50) NOT NULL,
    first_name VARCHAR(50),
    gender CHAR(1) NOT NULL CHECK (gender IN ('M', 'F', 'U')),
    primary_location_id INT REFERENCES location(location_id),
    race_cd CHAR(5) REFERENCES race(race_code),
    insurance_id INT REFERENCES insurance(insurance_id),
    provider_id INT REFERENCES healthcare_provider(provider_id)
);

-- 8. Diseased Patient
CREATE TABLE diseased_patient (
    person_id INT NOT NULL REFERENCES person(person_id),
    disease_id INT NOT NULL REFERENCES disease(disease_id),
    severity_value INT DEFAULT 1 CHECK (severity_value BETWEEN 1 AND 10),
    start_date DATE NOT NULL,
    end_date DATE,
    PRIMARY KEY (person_id, disease_id)
);

-- 9. Medicine
CREATE TABLE medicine (
    medicine_id SERIAL PRIMARY KEY,
    standard_industry_number VARCHAR(25),
    name VARCHAR(250) NOT NULL,
    company VARCHAR(150),
    active_ingredient_name VARCHAR(150)
);

-- 10. Indication
CREATE TABLE indication (
    medicine_id INT NOT NULL REFERENCES medicine(medicine_id),
    disease_id INT NOT NULL REFERENCES disease(disease_id),
    indication_date DATE,
    effectiveness_percent FLOAT CHECK (effectiveness_percent BETWEEN 0 AND 100),
    PRIMARY KEY (medicine_id, disease_id)
);

-- 11. Race Disease Propensity
CREATE TABLE race_disease_propensity (
    race_code CHAR(5) NOT NULL REFERENCES race(race_code),
    disease_id INT NOT NULL REFERENCES disease(disease_id),
    propensity_value INT CHECK (propensity_value BETWEEN 1 AND 10),
    PRIMARY KEY (race_code, disease_id)
);

-- 12. Test
CREATE TABLE test (
    test_id SERIAL PRIMARY KEY,
    person_id INT REFERENCES person(person_id),
    disease_id INT REFERENCES disease(disease_id),
    test_date DATE NOT NULL,
    result VARCHAR(50),
    result_value FLOAT
);
```

Populating Tables

```sql
-- 1. Disease Type
INSERT INTO disease_type (disease_type_code, disease_type_description, exclusions_other_note_text) VALUES
('INF', 'Infectious Disease', 'None'),
('CHR', 'Chronic Disease', 'Exclude mental disorders'),
('GEN', 'Genetic Disorder', 'Inherited conditions'),
('NUR', 'Neurological Disorder', 'Exclude trauma-induced'),
('CAN', 'Cancer', 'Malignant only'),
('MNT', 'Mental Health Disorder', 'Psychological conditions'),
('IMM', 'Immune System Disorder', 'Autoimmune types'),
('CAR', 'Cardiovascular Disease', 'Heart-related conditions'),
('RES', 'Respiratory Disease', 'Exclude allergy only'),
('END', 'Endocrine Disorder', 'Hormonal issues');

-- 2. Disease
INSERT INTO disease (disease_name, intensity_level_qty, disease_type_cd, source_disease_cd) VALUES
('COVID-19', 8, 'INF', NULL),
('Diabetes', 5, 'CHR', NULL),
('Tuberculosis', 7, 'INF', NULL),
('Breast Cancer', 9, 'CAN', NULL),
('Depression', 6, 'MNT', NULL),
('Asthma', 5, 'RES', NULL),
('Lupus', 7, 'IMM', NULL),
('Heart Attack', 9, 'CAR', NULL),
('Parkinson''s', 6, 'NUR', NULL),
('Thyroid Disorder', 4, 'END', NULL);

-- 3. Race
INSERT INTO race (race_code, race_description) VALUES
('ASN', 'Asian'),
('BLK', 'Black or African American'),
('WHT', 'White'),
('LAT', 'Latino/Hispanic'),
('NAT', 'Native American'),
('PAC', 'Pacific Islander'),
('MID', 'Middle Eastern'),
('MUL', 'Multiracial'),
('OTH', 'Other'),
('UND', 'Undisclosed');
```

Query | Query History

```sql
-- 4. Location
INSERT INTO location(city_name, state_province_name, country_name, developing_flag, wealth_rank_number) VALUES
('New York', 'NY', 'USA', 'N', 9),
('Lalitpur', 'Bagmati', 'Nepal', 'Y', 5),
('London', 'England', 'UK', 'N', 10),
('Delhi', 'Delhi', 'India', 'Y', 6),
('Tokyo', 'Tokyo', 'Japan', 'N', 10),
('Lagos', 'Lagos', 'Nigeria', 'Y', 3),
('Berlin', 'Berlin', 'Germany', 'N', 9),
('Rio', 'RJ', 'Brazil', 'Y', 6),
('Toronto', 'ON', 'Canada', 'N', 9),
('Sydney', 'NSW', 'Australia', 'N', 10);


-- 5. Insurance
INSERT INTO insurance (provider_name, plan_name, coverage_percent) VALUES
('Aetna', 'Silver Plan', 80),
('Blue Cross', 'Gold Plan', 90),
('United Health', 'Bronze Plan', 70),
('Cigna', 'Platinum Plan', 95),
('Humana', 'Basic Care', 75),
('Kaiser', 'Family Health', 85),
('Anthem', 'Essential Health', 88),
('WellCare', 'WellBasic', 73),
('Oscar', 'Core Plan', 78),
('Molina', 'Complete Care', 92);

-- 6. Healthcare Provider
INSERT INTO healthcare_provider (name, specialization, location_id) VALUES
('Dr. Smith', 'Internal Medicine', 1),
('Dr. Sharma', 'Pulmonologist', 2),
('Dr. Ahmed', 'Cardiologist', 3),
('Dr. Lee', 'Neurologist', 4),
('Dr. Mehta', 'Oncologist', 5),
('Dr. Chen', 'Endocrinologist', 6),
('Dr. Patel', 'General Practitioner', 7),
('Dr. Gomez', 'Psychiatrist', 8),
('Dr. Kim', 'Immunologist', 9),
('Dr. Singh', 'Family Medicine', 10);
```

```sql
-- 10. Diseased Patient
INSERT INTO diseased_patient (person_id, disease_id, severity_value, start_date, end_date) VALUES
(1, 1, 7, '2020-05-15', '2020-06-30'),
(2, 2, 6, '2019-03-10', NULL),
(3, 3, 8, '2021-11-05', NULL),
(4, 4, 9, '2022-02-01', NULL),
(5, 5, 5, '2021-01-20', NULL),
(6, 6, 6, '2020-12-10', NULL),
(7, 7, 7, '2021-03-15', NULL),
(8, 8, 8, '2022-07-07', NULL),
(9, 9, 6, '2023-05-01', NULL),
(10, 10, 5, '2021-09-25', NULL);

-- 11. Race Disease Propensity
INSERT INTO race_disease_propensity (race_code, disease_id, propensity_value) VALUES
('ASN', 2, 7),
('BLK', 2, 8),
('WHT', 1, 5),
('LAT', 5, 6),
('NAT', 4, 7),
('PAC', 3, 6),
('MID', 8, 8),
('MUL', 6, 7),
('OTH', 7, 5),
('UND', 9, 6);

-- 12. Test
INSERT INTO test (person_id, disease_id, test_date, result, result_value) VALUES
(1, 1, '2020-05-10', 'Positive', 1.0),
(2, 2, '2021-03-05', 'Negative', 0.0),
(3, 3, '2021-11-04', 'Positive', 1.0),
(4, 4, '2022-02-10', 'Positive', 1.0),
(5, 5, '2021-01-25', 'Negative', 0.0),
(6, 6, '2020-12-15', 'Positive', 1.0),
(7, 7, '2021-03-20', 'Negative', 0.0),
(8, 8, '2022-07-10', 'Positive', 1.0),
(9, 9, '2023-05-03', 'Negative', 0.0),
(10, 10, '2021-09-27', 'Positive', 1.0);
```

Query | Query History

```sql
-- 7. Person
INSERT INTO person (last_name, first_name, gender, primary_location_id, race_cd, insurance_id, provider_id) VALUES
('Lee', 'Min', 'F', 1, 'ASN', 1, 1),
('Jackson', 'Tina', 'F', 1, 'BLK', 2, 1),
('Bhandari', 'Raj', 'M', 2, 'ASN', 3, 2),
('Smith', 'John', 'M', 3, 'WHT', 4, 3),
('Khan', 'Sara', 'F', 4, 'MID', 5, 4),
('Perez', 'Luis', 'M', 5, 'LAT', 6, 5),
('Whitecloud', 'Maya', 'F', 6, 'NAT', 7, 6),
('Park', 'Jin', 'M', 7, 'PAC', 8, 7),
('Nguyen', 'Linh', 'F', 8, 'ASN', 9, 8),
('Brown', 'Alicia', 'F', 9, 'BLK', 10, 9);

-- 8. Medicine
INSERT INTO medicine (standard_industry_number, name, company, active_ingredient_name) VALUES
('SIN123', 'Remdesivir', 'Gilead', 'GS-5734'),
('SIN456', 'Metformin', 'Sun Pharma', 'Metformin Hydrochloride'),
('SIN789', 'Aspirin', 'Bayer', 'Acetylsalicylic Acid'),
('SIN101', 'Lipitor', 'Pfizer', 'Atorvastatin'),
('SIN202', 'Prozac', 'Eli Lilly', 'Fluoxetine'),
('SIN303', 'Prednisone', 'Teva', 'Prednisone'),
('SIN404', 'Lisinopril', 'Novartis', 'Lisinopril'),
('SIN505', 'Tamoxifen', 'AstraZeneca', 'Tamoxifen Citrate'),
('SIN606', 'Levothyroxine', 'AbbVie', 'Levothyroxine'),
('SIN707', 'Insulin', 'Novo Nordisk', 'Insulin Human');

-- 9. Indication
INSERT INTO indication (medicine_id, disease_id, indication_date, effectiveness_percent) VALUES
(1, 1, '2020-06-01', 65.0),
(2, 2, '2018-01-15', 78.5),
(3, 8, '2021-02-01', 84.2),
(4, 8, '2020-03-21', 89.0),
(5, 5, '2022-05-10', 60.0),
(6, 7, '2021-07-14', 74.6),
(7, 8, '2019-09-30', 77.5),
(8, 4, '2021-11-01', 81.1),
(9, 10, '2022-03-10', 68.0),
(10, 2, '2017-10-05', 88.9);
```

Operational Queries and Data Manipulation

Query | Query History

```sql
-- 1. List all patients and the diseases they have
SELECT p.first_name, p.last_name, d.disease_name, dp.severity_value
FROM diseased_patient dp
JOIN person p ON dp.person_id = p.person_id
JOIN disease d ON dp.disease_id = d.disease_id;
```

Data Output | Messages | Notifications

	first_name character varying (50)	last_name character varying (50)	disease_name character varying (100)	severity_value integer
1	Min	Lee	COVID-19	7
2	Tina	Jackson	Diabetes	6
3	Raj	Bhandari	Tuberculosis	8
4	John	Smith	Breast Cancer	9
5	Sara	Khan	Depression	5
6	Luis	Perez	Asthma	6
7	Maya	Whitecloud	Lupus	7
8	Jin	Park	Heart Attack	8
9	Linh	Nguyen	Parkinson's	6
10	Alicia	Brown	Thyroid Disorder	5

```sql
-- 2. Find the most common diseases by race
SELECT r.race_description, d.disease_name, COUNT(*) AS case_count
FROM diseased_patient dp
JOIN person p ON dp.person_id = p.person_id
JOIN disease d ON dp.disease_id = d.disease_id
JOIN race r ON p.race_cd = r.race_code
GROUP BY r.race_description, d.disease_name
ORDER BY case_count DESC;
```

Data Output | Messages | Notifications

	race_description character varying (100)	disease_name character varying (100)	case_count bigint
1	Native American	Lupus	1
2	Asian	COVID-19	1
3	White	Breast Cancer	1
4	Black or African American	Diabetes	1
5	Middle Eastern	Depression	1
6	Asian	Tuberculosis	1
7	Asian	Parkinson's	1
8	Pacific Islander	Heart Attack	1
9	Black or African American	Thyroid Disorder	1
10	Latino/Hispanic	Asthma	1

```sql
-- 3. Get medicine effectiveness for each disease
SELECT d.disease_name, m.name AS medicine_name, i.effectiveness_percent
FROM indication i
JOIN disease d ON i.disease_id = d.disease_id
JOIN medicine m ON i.medicine_id = m.medicine_id;
```

Data Output | Messages | Notifications

	disease_name character varying (100)	medicine_name character varying (250)	effectiveness_percent double precision
1	COVID-19	Remdesivir	65
2	Diabetes	Metformin	78.5
3	Heart Attack	Aspirin	84.2
4	Heart Attack	Lipitor	89
5	Depression	Prozac	60
6	Lupus	Prednisone	74.6
7	Heart Attack	Lisinopril	77.5
8	Breast Cancer	Tamoxifen	81.1
9	Thyroid Disorder	Levothyroxine	68
10	Diabetes	Insulin	88.9

```sql
-- 4. Find patients who tested positive for any disease
SELECT p.first_name, p.last_name, d.disease_name, t.test_date
FROM test t
JOIN person p ON t.person_id = p.person_id
JOIN disease d ON t.disease_id = d.disease_id
WHERE t.result = 'Positive';
```

Data Output | Messages | Notifications

	first_name character varying (50)	last_name character varying (50)	disease_name character varying (100)	test_date date
1	Min	Lee	COVID-19	2020-05-10
2	Raj	Bhandari	Tuberculosis	2021-11-04
3	John	Smith	Breast Cancer	2022-02-10
4	Luis	Perez	Asthma	2020-12-15
5	Jin	Park	Heart Attack	2022-07-10
6	Alicia	Brown	Thyroid Disorder	2021-09-27

DML OPERATIONS

Query Query History

```
1  -- Insert a new test for an existing patient
2  INSERT INTO test (person_id, disease_id, test_date, result, result_value)
3  VALUES (1, 2, '2024-01-01', 'Negative', 0.0);
```

Data Output Messages Notifications

```
INSERT 0 1

Query returned successfully in 104 msec.
```

```
5  -- Update severity of a disease case
6  UPDATE diseased_patient
7  SET severity_value = 9
8  WHERE person_id = 2 AND disease_id = 2;
9
```

Data Output Messages Notifications

```
UPDATE 1

Query returned successfully in 89 msec.
```

```
10 -- Try deleting a medicine (will fail if it is referenced in indication table)
11 DELETE FROM medicine
12 WHERE medicine_id = 1;
13
```

Data Output Messages Notifications

```
ERROR:  Key (medicine_id)=(1) is still referenced from table "indication".update or delete on table "medicine" violates foreign key constraint "indication_medicine_id_fkey" on table "indication"

ERROR:  update or delete on table "medicine" violates foreign key constraint "indication_medicine_id_fkey" on table "indication"
SQL state: 23503
Detail: Key (medicine_id)=(1) is still referenced from table "indication".
```

```
14 -- Delete test data for a patient (if allowed)
15 DELETE FROM test
16 WHERE person_id = 3;
17
```

Data Output Messages Notifications

```
DELETE 1

Query returned successfully in 86 msec.
```

Dimensional Modeling with Star Schema

A star schema dimensional model is created to support analytical queries and business intelligence. Dimension tables capture attributes like patient demographics and disease details.

Slowly Changing Dimension Type 2 (SCD2) is implemented to track historical changes in dimension data, preserving data accuracy over time.

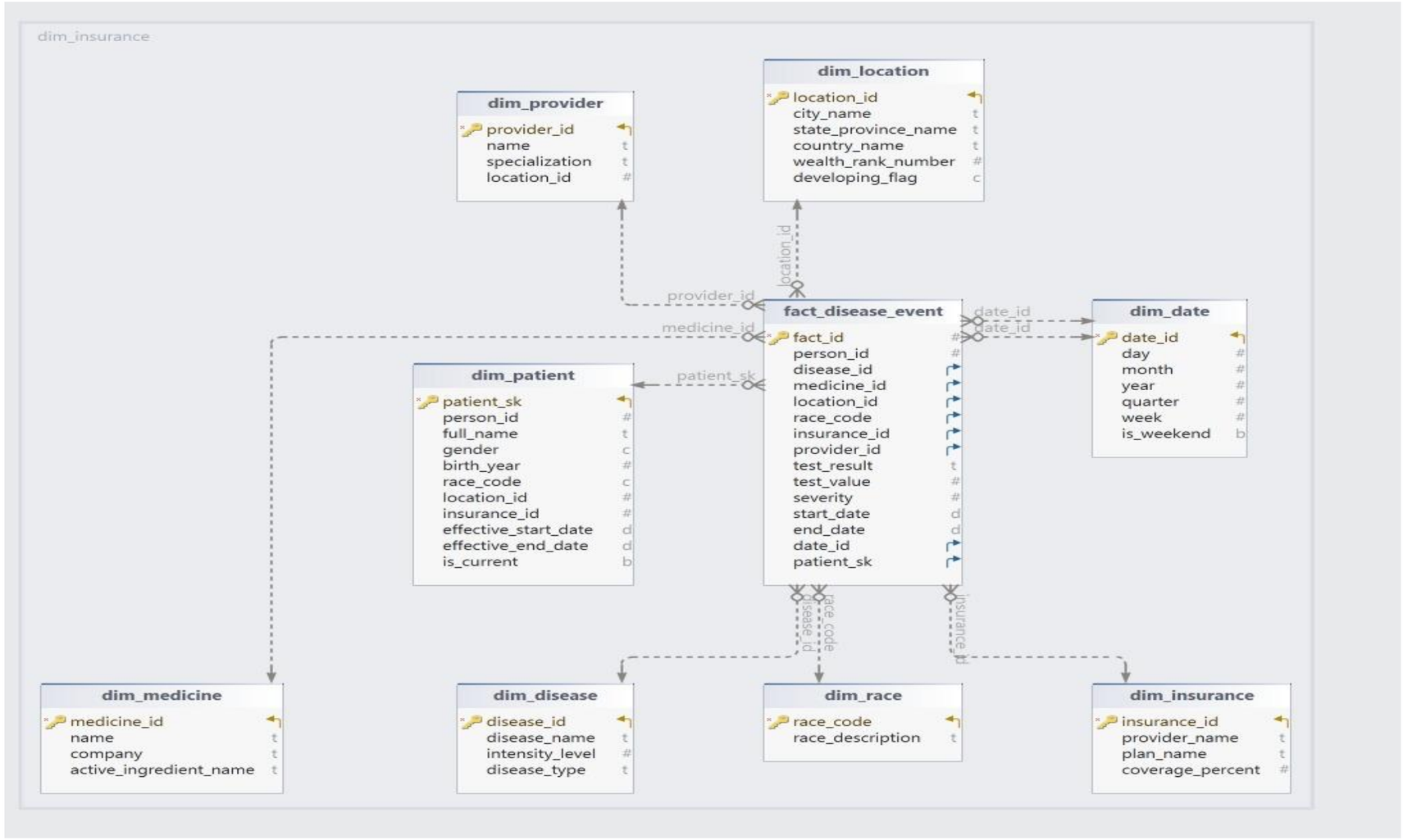
Dimension Tables

Store descriptive attributes for analysis.

SCD Type 2

Maintains historical versions of dimension records.

Dimensional Modeling- Disease DW



Creating and Loading Dimensions

```sql

-- Create schema for Data Warehouse
CREATE SCHEMA IF NOT EXISTS disease_dwh;
SET search_path TO disease_dwh;

-- 1. dim_patient
CREATE TABLE dim_patient (
    patient_id SERIAL PRIMARY KEY,
    person_id INT,
    full_name VARCHAR(100),
    gender CHAR(1),
    birth_year INT,
    race_code CHAR(5),
    location_id INT
);

-- 2. dim_disease
CREATE TABLE dim_disease (
    disease_id INT PRIMARY KEY,
    disease_name VARCHAR(100),
    intensity_level INT,
    disease_type VARCHAR(100)
);

-- 3. dim_medicine
CREATE TABLE dim_medicine (
    medicine_id INT PRIMARY KEY,
    name VARCHAR(250),
    company VARCHAR(150),
    active_ingredient_name VARCHAR(150)
);

```

Query Query History

```sql
-- 4. dim_location
CREATE TABLE dim_location (
    location_id INT PRIMARY KEY,
    city_name VARCHAR(100),
    state_province_name VARCHAR(100),
    country_name VARCHAR(100),
    wealth_rank_number INT,
    developing_flag CHAR(1)
);

-- 5. dim_race
CREATE TABLE dim_race (
    race_code CHAR(5) PRIMARY KEY,
    race_description VARCHAR(100)
);

-- 6. dim_insurance
CREATE TABLE dim_insurance (
    insurance_id INT PRIMARY KEY,
    provider_name VARCHAR(150),
    plan_name VARCHAR(150),
    coverage_percent FLOAT
);

-- 7. dim_provider
CREATE TABLE dim_provider (
    provider_id INT PRIMARY KEY,
    name VARCHAR(150),
    specialization VARCHAR(100),
    location_id INT
);

```

```sql
-- 8. dim_date
CREATE TABLE disease_dwh.dim_date (
    date_id DATE PRIMARY KEY,
    day INT,
    month INT,
    year INT,
    quarter INT,
    week INT,
    is_weekend BOOLEAN
);

-- 9. fact_disease_event
CREATE TABLE disease_dwh.fact_disease_event (
    fact_id SERIAL PRIMARY KEY,
    person_id INT,
    disease_id INT,
    medicine_id INT,
    location_id INT,
    race_code CHAR(5),
    insurance_id INT,
    provider_id INT,
    test_result VARCHAR(50),
    test_value FLOAT,
    severity INT,
    start_date DATE,
    end_date DATE,
    date_id DATE REFERENCES disease_dwh.dim_date(date_id) -- FK to dim_date
);

```

```sql

-- ETL Script: Populate Dimension Tables

-- 1. dim_patient
INSERT INTO disease_dwh.dim_patient (person_id, full_name, gender, birth_year, race_code, location_id)
SELECT
    person_id,
    CONCAT(first_name, ' ', last_name),
    gender,
    EXTRACT(YEAR FROM CURRENT_DATE) - 30,  -- Assume everyone is 30 years old for mock data
    race_cd,
    primary_location_id
FROM person;

-- 2. dim_disease
INSERT INTO disease_dwh.dim_disease (disease_id, disease_name, intensity_level, disease_type)
SELECT
    d.disease_id,
    d.disease_name,
    d.intensity_level_qty,
    dt.disease_type_description
FROM disease d
JOIN disease_type dt ON d.disease_type_cd = dt.disease_type_code;

-- 3. dim_medicine
INSERT INTO disease_dwh.dim_medicine (medicine_id, name, company, active_ingredient_name)
SELECT medicine_id, name, company, active_ingredient_name
FROM medicine;

-- 4. dim_location
INSERT INTO disease_dwh.dim_location (location_id, city_name, state_province_name, country_name, wealth_rank_number, developing_flag)
SELECT location_id, city_name, state_province_name, country_name, wealth_rank_number, developing_flag
FROM location;

-- 5. dim_race
INSERT INTO disease_dwh.dim_race (race_code, race_description)
SELECT race_code, race_description
FROM race;

```

Creating and Loading Dimensions (cont)

```sql
-- 6. dim_insurance
INSERT INTO disease_dwh.dim_insurance (insurance_id, provider_name, plan_name, coverage_percent)
SELECT insurance_id, provider_name, plan_name, coverage_percent
FROM insurance;

-- 7. dim_provider
INSERT INTO disease_dwh.dim_provider (provider_id, name, specialization, location_id)
SELECT provider_id, name, specialization, location_id
FROM healthcare_provider;

-- Generate full date range in dim_date (2020-01-01 to 2025-12-31)
-- Extend dim_date to start from 2015
INSERT INTO disease_dwh.dim_date (date_id, day, month, year, quarter, week, is_weekend)
SELECT
    d::DATE,
    EXTRACT(DAY FROM d),
    EXTRACT(MONTH FROM d),
    EXTRACT(YEAR FROM d),
    EXTRACT(QUARTER FROM d),
    EXTRACT(WEEK FROM d),
    CASE WHEN EXTRACT(DOW FROM d) IN (0, 6) THEN TRUE ELSE FALSE END
FROM generate_series('2015-01-01'::DATE, '2019-12-31'::DATE, '1 day'::INTERVAL) d;


SELECT COUNT(*) FROM disease_dwh.dim_date;

```

```sql
-- 8. fact_disease_event
INSERT INTO disease_dwh.fact_disease_event (
    person_id, disease_id, medicine_id, location_id, race_code,
    insurance_id, provider_id, test_result, test_value, severity,
    start_date, end_date, date_id
)
SELECT
    p.person_id,
    dp.disease_id,
    i.medicine_id,
    p.primary_location_id,
    p.race_cd,
    p.insurance_id,
    p.provider_id,
    t.result,
    t.result_value,
    dp.severity_value,
    dp.start_date,
    dp.end_date,
    dp.start_date
FROM diseased_patient dp
JOIN person p ON dp.person_id = p.person_id
LEFT JOIN test t ON t.person_id = dp.person_id AND t.disease_id = dp.disease_id
LEFT JOIN indication i ON i.disease_id = dp.disease_id;

```

SCD -Type 2

```sql
DROP TABLE IF EXISTS disease_dwh.dim_patient;

CREATE TABLE disease_dwh.dim_patient (
    patient_sk SERIAL PRIMARY KEY,              -- Surrogate key
    person_id INT,                              -- Business key
    full_name VARCHAR(100),
    gender CHAR(1),
    birth_year INT,
    race_code CHAR(5),
    location_id INT,
    insurance_id INT,
    effective_start_date DATE,                  -- SCD2 tracking
    effective_end_date DATE,
    is_current BOOLEAN
);

SELECT * FROM disease_dwh.dim_patient;
```

Data Output | Messages | Notifications

patient_sk [PK] integer	person_id integer	full_name character varying (100)	gender character (1)	birth_year integer	race_code character (5)	location_id integer	insurance_id integer	effective_start_date date	effective_end_date date	is_current boolean

```sql
INSERT INTO disease_dwh.dim_patient (person_id, full_name, gender, birth_year, race_code, location_id)
SELECT
    person_id,
    CONCAT(first_name, ' ', last_name),
    gender,
    EXTRACT(YEAR FROM CURRENT_DATE) - 30,  -- Assume everyone is 30 years old for mock data
    race_cd,
    primary_location_id
FROM person;

Select * from disease_dwh.dim_patient;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10 | Page No: 1

patient_sk [PK] integer	person_id integer	full_name character varying (100)	gender character (1)	birth_year integer	race_code character (5)	location_id integer	insurance_id integer	effective_start_date date	effective_end_date date	is_current boolean
1	1	Min Lee	F	1995	ASN	1	[null]	[null]	[null]	[null]
2	2	Tina Jackson	F	1995	BLK	1	[null]	[null]	[null]	[null]
3	3	Raj Bhandari	M	1995	ASN	2	[null]	[null]	[null]	[null]
4	4	John Smith	M	1995	WHT	3	[null]	[null]	[null]	[null]
5	5	Sara Khan	F	1995	MID	4	[null]	[null]	[null]	[null]
6	6	Luis Perez	M	1995	LAT	5	[null]	[null]	[null]	[null]
7	7	Maya Whitecloud	F	1995	NAT	6	[null]	[null]	[null]	[null]
8	8	Jin Park	M	1995	PAC	7	[null]	[null]	[null]	[null]
9	9	Linh Nguyen	F	1995	ASN	8	[null]	[null]	[null]	[null]
10	10	Alicia Brown	F	1995	BLK	9	[null]	[null]	[null]	[null]

```sql
INSERT INTO disease_dwh.dim_patient (
    person_id, full_name, gender, birth_year, race_code, location_id,
    insurance_id, effective_start_date, effective_end_date, is_current
)
VALUES (
    3, 'Raj Bhandari', 'M', 1995, 'ASN', 5, 5,
    CURRENT_DATE, NULL, TRUE
);

Select * from disease_dwh.dim_patient;
```

Data Output | Messages | Notifications

Showing rows: 1 to 11 | Page No: 1

patient_sk [PK] integer	person_id integer	full_name character varying (100)	gender character (1)	birth_year integer	race_code character (5)	location_id integer	insurance_id integer	effective_start_date date	effective_end_date date	is_current boolean
1	1	Min Lee	F	1995	ASN	1	[null]	[null]	[null]	[null]
2	2	Tina Jackson	F	1995	BLK	1	[null]	[null]	[null]	[null]
3	4	John Smith	M	1995	WHT	3	[null]	[null]	[null]	[null]
4	5	Sara Khan	F	1995	MID	4	[null]	[null]	[null]	[null]
5	6	Luis Perez	M	1995	LAT	5	[null]	[null]	[null]	[null]
6	7	Maya Whitecloud	F	1995	NAT	6	[null]	[null]	[null]	[null]
7	8	Jin Park	M	1995	PAC	7	[null]	[null]	[null]	[null]
8	9	Linh Nguyen	F	1995	ASN	8	[null]	[null]	[null]	[null]
9	10	Alicia Brown	F	1995	BLK	9	[null]	[null]	[null]	[null]
10	3	Raj Bhandari	M	1995	ASN	2	[null]	2025-05-01	false	
11	3	Raj Bhandari	M	1995	ASN	5	5	2025-05-02	[null]	true

Analytical Queries on Star

```sql
-- Sample Analytical Queries (Star Schema)
--- 1.  Top 5 most common diseases by Case Count
SELECT d.disease_name, COUNT(*) AS case_count
FROM disease_dwh.fact_disease_event f
JOIN disease_dwh.dim_disease d ON f.disease_id = d.disease_id
GROUP BY d.disease_name
ORDER BY case_count DESC
LIMIT 5;
```

Data Output | Messages | Notifications

	disease_name character varying (100)	case_count bigint
1	Heart Attack	3
2	Diabetes	2
3	Parkinson's	1
4	Tuberculosis	1
5	Breast Cancer	1

```sql
---4. Disease Count by Insurance plan
SELECT i.plan_name, COUNT(*) AS total_cases
FROM disease_dwh.fact_disease_event f
JOIN disease_dwh.dim_insurance i ON f.insurance_id = i.insurance_id
GROUP BY i.plan_name
ORDER BY total_cases DESC;
```

Data Output | Messages | Notifications

	plan_name character varying (150)	total_cases bigint
1	WellBasic	3
2	Gold Plan	2
3	Core Plan	1
4	Basic Care	1
5	Family Health	1
6	Platinum Plan	1
7	Complete Care	1
8	Essential Health	1
9	Bronze Plan	1
10	Silver Plan	1

```sql
---3. Positive Test Rates by Race
SELECT r.race_description, COUNT(*) FILTER (WHERE f.test_result = 'Positive')::float / COUNT(*) AS positive_rate
FROM disease_dwh.fact_disease_event f
JOIN disease_dwh.dim_race r ON f.race_code = r.race_code
GROUP BY r.race_description
ORDER BY positive_rate DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to

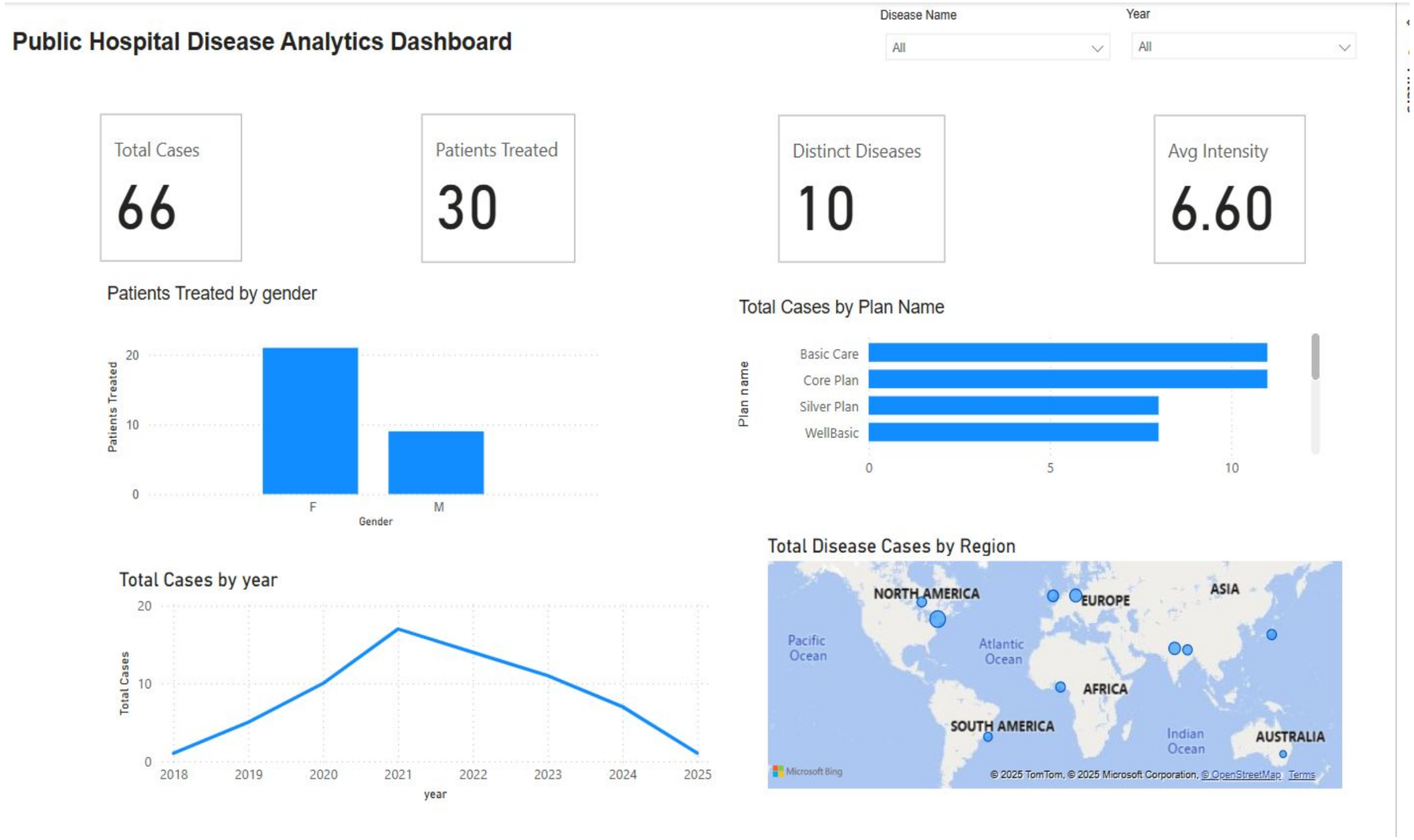
	race_description character varying (100)	positive_rate double precision
1	Latino/Hispanic	1
2	Pacific Islander	1
3	White	1
4	Asian	0.3333333333333333
5	Black or African American	0.3333333333333333
6	Middle Eastern	0
7	Native American	0

```sql
---5. Monthly Case Trends
SELECT dd.year, dd.month, COUNT(*) AS monthly_cases
FROM disease_dwh.fact_disease_event f
JOIN disease_dwh.dim_date dd ON f.date_id = dd.date_id
GROUP BY dd.year, dd.month
ORDER BY dd.year, dd.month;
```

Data Output | Messages | Notifications

	year integer	month integer	monthly_cases bigint
1	2019	3	2
2	2020	5	1
3	2020	12	1
4	2021	1	1
5	2021	3	1
6	2021	9	1
7	2021	11	1
8	2022	2	1
9	2022	7	3
10	2023	5	1

POWER BI DASHBOARD



NoSQL Database Comparison

Comparing relational PostgreSQL with NoSQL alternatives reveals trade-offs. MongoDB stores embedded documents, simplifying individual patient data retrieval but risking duplication and complex analytics challenges.

Neo4j graph database excels at modeling relationships like contact tracing and disease transmission but is less suited for tabular analytics and requires specialized query languages.

MongoDB

- Embedded documents for patient-centric data
- Flexible schema, real-time applications
- Challenges with data duplication and analytics

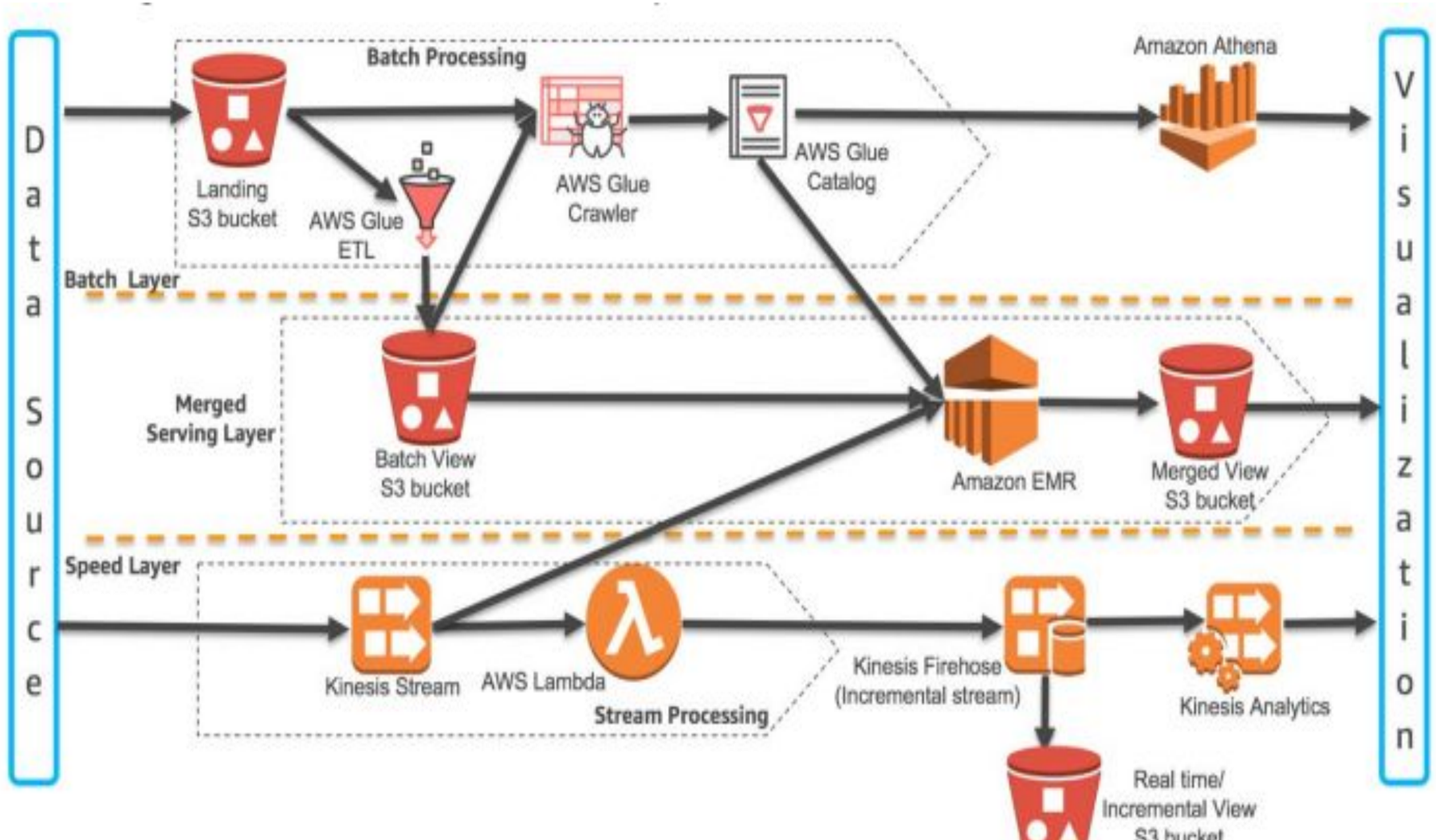
Neo4j

- Graph model for relationships and contact tracing
- Pattern matching with Cypher query language
- Limited BI tool support

```
(:Person {name: 'Raj'})
  -[:HAS_DISEASE {severity: 6}]-> (:Disease {name: 'Diabetes'})
  -[:LIVES_IN]-> (:Location {city: 'Kathmandu'})
  -[:COVERED_BY]-> (:Insurance {plan_name: 'Gold Plan'})
  -[:VISITED]-> (:Provider {name: 'Dr. Sharma'})
  -[:TESTED_FOR]-> (:Test {result: 'Positive'})
```

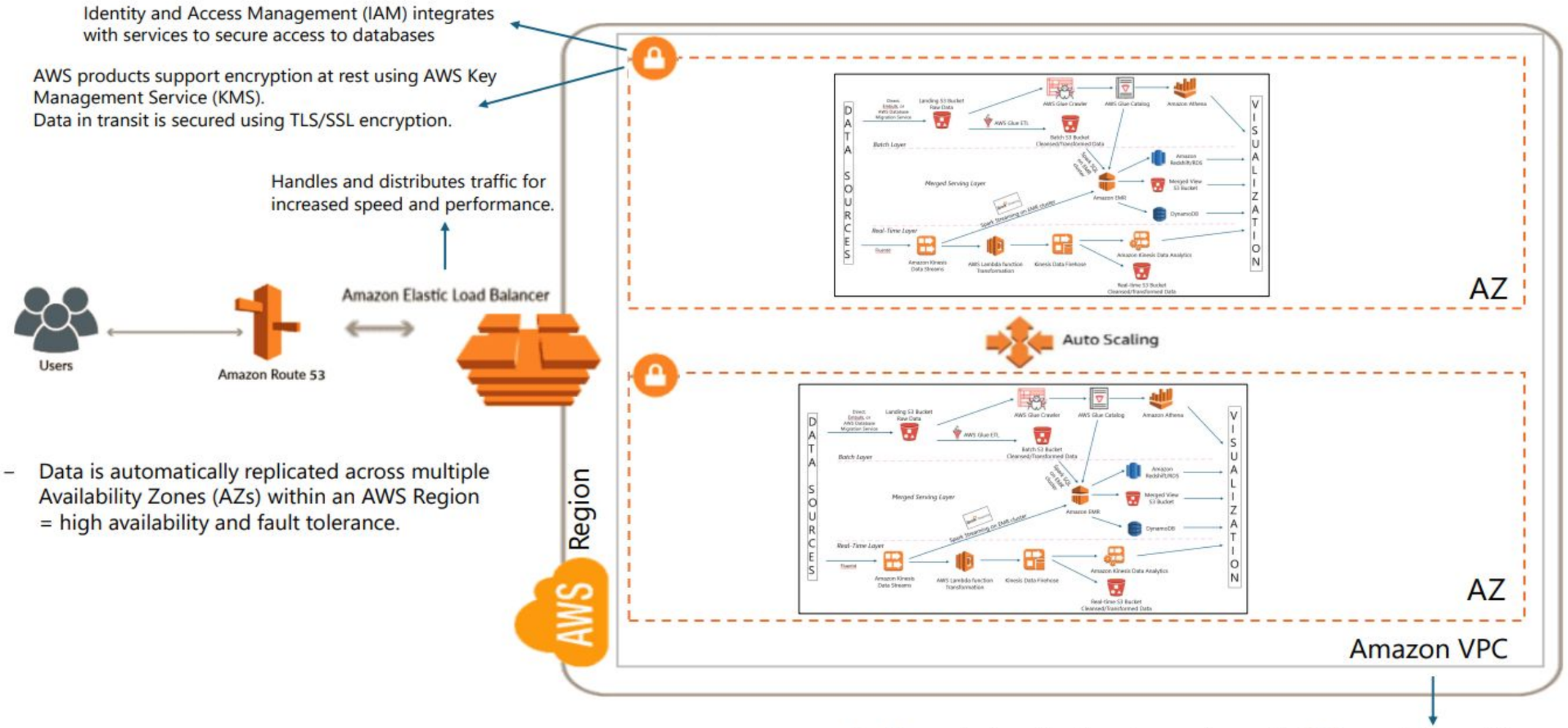

```
{
  "person_id": 1,
  "name": "Min Lee",
  "gender": "F",
  "location": {
    "city": "New York",
    "country": "USA"
  },
  "insurance": {
    "provider_name": "Aetna",
    "plan_name": "Silver Plan"
  },
  "diseases": [
    {
      "disease_name": "COVID-19",
      "severity": 7,
      "start_date": "2020-05-15",
      "test": {
        "result": "Positive",
        "value": 1.0,
        "test_date": "2020-05-10"
      }
    }
  ]
}
```

AWS Lambda Architecture for Data Processing



AWS Lambda Architecture for Data (cont)

Processing



Snowflake Advantages for Healthcare Analytics

Snowflake offers a cloud data warehouse with automatic performance tuning, elastic scalability, and separation of storage and compute. It supports concurrent dashboards and real-time data integration, ideal for large-scale healthcare analytics.

This platform complements PostgreSQL by handling complex analytical workloads efficiently, enabling secure data sharing and real-time updates for disease trend monitoring.

Performance & Scalability

Automatic tuning and consistent query speed at scale.

Real-Time & Concurrent Access

Supports simultaneous users and live data ingestion.

Conclusion

- The Disease Data Management and Analytics System enhances public health outcomes by leveraging a robust relational database model and a star schema for analytical insights.
- Incorporates Slowly Changing Dimension Type 2 (SCD2) to maintain historical data accuracy.
- Uses AWS Lambda architecture to facilitate scalable, real-time data processing.
- Integrates Snowflake for efficient handling of large-scale healthcare data with performance optimization.
- Compares relational databases with NoSQL alternatives to balance structured data management and flexible, real-time analytics.
- Supports informed decision-making for public health agencies through comprehensive data management and analytics.